Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт
До лабораторної роботи №5
Варіант 9
На тему
“Ліниві обчислення: генератори та itertools”

Виконав:
Студент групи ФЕП-22с
Обиход І. В.
Перевірив:
Ас. Остріков Д. А.

Львів 2026

Мета роботи:

- зрозуміти generator expressions, yield/yield from, ітератори та лінивість;
- уміти працювати з нескінченними послідовностями;
- освоїти islice, takewhile, dropwhile, accumulate, count, cycle, repeat;
- реалізувати нескінченний стрім Фібоначчі, sliding window, take/drop/accumulate.

Теоретичні відомості:

- Ітератор - об'єкт зі станом, що на кожному кроці повертає наступне значення і завершується StopIteration.
- Лінивість: значення обчислюються в міру запиту, що економить пам'ять і дозволяє працювати з нескінченними джерелами.
- Генераторні вирази та функції з yield створюють ліниві послідовності; генератор вичерпується після одного проходу.
- itertools надає готові блоки: islice, takewhile, dropwhile, accumulate, count, cycle, repeat.

Завдання (варіант 9 - Циклічний розклад):

Побудувати нескінченний циклічний розклад зі списку змін/чергувань та отримати план на N днів; додати нумерацію. Інструменти: cycle, count, zip, islice.

Хід роботи

Базові утиліти реалізовано в streams.py: нескінченні генератори fib_stream і naturals, take і drop на основі islice, sliding_window через deque(maxlen=k), moving_average, лінивий chunked, обгортки над accumulate, takewhile та dropwhile. Некоректні розміри вікна чи від'ємні n дають ValueError.

Варіант 9 реалізовано в schedule.py. cyclic_schedule повертає нескінченний потік днів Day: нумерацію дає count, дні тижня і зміни циклічно повторюються через cycle, а zip об'єднує ці потоки. plan бере перші N днів через islice.

Додатково реалізовано duty_roster (працівники і зміни чергуються незалежно), days_until (дні до першої потрібної зміни через takewhile), weeks (групування по 7 днів через chunked) і cumulative_hours (накопичені години через accumulate).

Нескінченні потоки ніде не матеріалізуються повністю: список створюється лише для запитаних N елементів.

Тести перевіряють правильність розкладу, циклічність, крайні випадки (n=0, порожнє джерело, k більше за довжину) і лінивість: з джерела змін береться рівно стільки елементів, скільки потрібно.

Файл schedule.py

schedule.py рядки 1-60

```python
"""Варіант 9 ЛР5: нескінченний циклічний розклад змін.

Розклад будується ліниво з cycle, count, zip та islice. Значення
обчислюються лише тоді, коли їх запитують.
"""

from collections.abc import Iterable, Iterator, Mapping
from itertools import accumulate, count, cycle, islice, takewhile
from typing import NamedTuple

from streams import chunked, take

WEEKDAYS: tuple[str, ...] = ("Пн", "Вт", "Ср", "Чт", "Пт", "Сб", "Нд")


class Day(NamedTuple):
    """Один день розкладу."""

    number: int
    weekday: str
    shift: str


class Duty(NamedTuple):
    """Чергування працівника в конкретний день."""

    number: int
    worker: str
    shift: str


def weekdays_from(start_weekday: int = 0) -> Iterator[str]:
    """Нескінченний потік днів тижня, починаючи з заданого (0 - понеділок)."""
    if not 0 <= start_weekday < len(WEEKDAYS):
        raise ValueError("start_weekday має бути в межах 0..6")
    return islice(cycle(WEEKDAYS), start_weekday, None)


def cyclic_schedule(shifts: Iterable[str], start_weekday: int = 0) -> Iterator[Day]:
    """Нескінченний розклад: нумерація count, дні тижня і зміни через cycle."""
    days = weekdays_from(start_weekday)
    return (Day(n, wd, s) for n, wd, s in zip(count(1), days, cycle(shifts)))


def plan(shifts: Iterable[str], days: int, start_weekday: int = 0) -> list[Day]:
    """План на N днів (матеріалізуються лише N елементів)."""
    return take(days, cyclic_schedule(shifts, start_weekday))


def duty_roster(workers: Iterable[str], shifts: Iterable[str]) -> Iterator[Duty]:
    """Нескінченний графік чергувань: працівники і зміни чергуються циклічно."""
    return (Duty(n, w, s) for n, w, s in zip(count(1), cycle(workers), cycle(shifts)))


def days_until(shifts: Iterable[str], target: str, limit: int = 366) -> list[Day]:
    """Дні до першої появи зміни target (takewhile), не більше limit днів."""
    schedule = islice(cyclic_schedule(shifts), limit)
    return list(takewhile(lambda d: d.shift != target, schedule))
```

schedule.py рядки 61-70

```python
def weeks(shifts: Iterable[str], n_weeks: int) -> list[tuple[Day, ...]]:
    """Розклад, згрупований по тижнях (ліниві пакети по 7 днів)."""
    return take(n_weeks, chunked(cyclic_schedule(shifts), 7))


def cumulative_hours(
    shifts: Iterable[str], hours: Mapping[str, float], days: int
) -> list[float]:
    """Накопичені робочі години за N днів через accumulate."""
    return list(accumulate(hours.get(d.shift, 0.0) for d in plan(shifts, days)))
```

Файл streams.py

streams.py рядки 1-60

```python
"""Базові ліниві утиліти ЛР5: Фібоначчі, take/drop, sliding window, accumulate."""

import operator
from collections import deque
from collections.abc import Iterable, Iterator
from itertools import accumulate, count, dropwhile, islice, takewhile


def fib_stream() -> Iterator[int]:
    """Нескінченний потік чисел Фібоначчі."""
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b


def naturals(start: int = 1) -> Iterator[int]:
    """Нескінченний лічильник натуральних чисел."""
    yield from count(start)


def take[T](n: int, it: Iterable[T]) -> list[T]:
    """Бере перші n елементів (матеріалізує лише їх)."""
    if n < 0:
        raise ValueError("n має бути невід'ємним")
    return list(islice(it, n))


def drop[T](n: int, it: Iterable[T]) -> Iterator[T]:
    """Пропускає перші n елементів і повертає ітератор на решту."""
    if n < 0:
        raise ValueError("n має бути невід'ємним")
    iterator = iter(it)
    next(islice(iterator, n, n), None)
    return iterator


def sliding_window[T](it: Iterable[T], k: int) -> Iterator[tuple[T, ...]]:
    """Ліниві ковзні вікна довжини k через deque(maxlen=k)."""
    if k <= 0:
        raise ValueError("Розмір вікна має бути додатним")
    return _windows(iter(it), k)


def _windows[T](iterator: Iterator[T], k: int) -> Iterator[tuple[T, ...]]:
    """Генератор вікон: спершу наповнює вікно, далі зсуває по одному."""
    window: deque[T] = deque(islice(iterator, k), maxlen=k)
    if len(window) < k:
        return
    yield tuple(window)
    for x in iterator:
        window.append(x)
        yield tuple(window)


def moving_average(it: Iterable[float], k: int) -> Iterator[float]:
    """Рухоме середнє по вікнах довжини k."""
    return (sum(w) / k for w in sliding_window(it, k))
```

streams.py рядки 61-91

```python
def chunked[T](it: Iterable[T], k: int) -> Iterator[tuple[T, ...]]:
    """Ліниві пакети фіксованого розміру (останній може бути коротшим)."""
    if k <= 0:
        raise ValueError("Розмір пакета має бути додатним")
    return _chunks(iter(it), k)


def _chunks[T](iterator: Iterator[T], k: int) -> Iterator[tuple[T, ...]]:
    """Генератор пакетів через islice."""
    while chunk := tuple(islice(iterator, k)):
        yield chunk


def prefix_sums(xs: Iterable[float]) -> Iterator[float]:
    """Префіксні суми через accumulate."""
    return accumulate(xs)


def prefix_products(xs: Iterable[int]) -> Iterator[int]:
    """Префіксні добутки через accumulate з operator.mul."""
    return accumulate(xs, operator.mul)


def below(limit: float, it: Iterable[float]) -> Iterator[float]:
    """Бере значення, поки вони менші за поріг (takewhile)."""
    return takewhile(lambda x: x < limit, it)


def after_non_positive(it: Iterable[float]) -> Iterator[float]:
    """Пропускає початкові недодатні значення (dropwhile)."""
    return dropwhile(lambda x: x <= 0, it)
```

Файл main.py

main.py рядки 1-41

```python
"""Точка входу лабораторної роботи №5, варіант 9."""

from itertools import islice

from schedule import cumulative_hours, days_until, duty_roster, plan, weeks
from streams import drop, fib_stream, moving_average, naturals, sliding_window, take

SHIFTS = ("Ранок", "День", "Ніч", "Вихідний")
HOURS = {"Ранок": 8.0, "День": 8.0, "Ніч": 10.0, "Вихідний": 0.0}
WORKERS = ("Іван", "Олена", "Петро")


def run() -> None:
    """Демонструє лінивий циклічний розклад і базові потоки."""
    print("План на 10 днів (старт у середу):")
    for day in plan(SHIFTS, 10, start_weekday=2):
        print(f"  день {day.number:>2} ({day.weekday}): {day.shift}")

    print("\nГрафік чергувань на 6 днів:")
    for duty in islice(duty_roster(WORKERS, SHIFTS), 6):
        print(f"  день {duty.number}: {duty.worker} - {duty.shift}")

    print(f"\nДнів до першої нічної зміни: {len(days_until(SHIFTS, 'Ніч'))}")
    print(f"Накопичені години за 8 днів: {cumulative_hours(SHIFTS, HOURS, 8)}")
    first_week = weeks(SHIFTS, 1)[0]
    print(f"Перший тиждень: {[d.shift for d in first_week]}")

    print("\nБазові потоки:")
    print(f"  перші 10 Фібоначчі: {take(10, fib_stream())}")
    print(f"  пропустити 5, взяти 8: {take(8, drop(5, fib_stream()))}")
    print(f"  вікна по 3: {take(3, sliding_window(naturals(), 3))}")
    print(f"  рухоме середнє k=2: {take(4, moving_average(naturals(), 2))}")


def main() -> None:
    """Запуск демонстрації лінивого розкладу."""
    run()


if __name__ == "__main__":
    main()
```

Файл tests/test_schedule.py

test_schedule.py рядки 1-57

```python
"""Тести варіанта 9: циклічний розклад."""

from collections.abc import Iterator
from itertools import islice

import pytest

from schedule import (
    Day,
    cumulative_hours,
    cyclic_schedule,
    days_until,
    duty_roster,
    plan,
    weekdays_from,
    weeks,
)

SHIFTS = ("A", "B", "C")


def test_plan_numbering_and_cycle() -> None:
    """Тест: нумерація днів, дні тижня і циклічність змін."""
    days = plan(SHIFTS, 5)
    assert [d.number for d in days] == [1, 2, 3, 4, 5]
    assert [d.shift for d in days] == ["A", "B", "C", "A", "B"]
    assert [d.weekday for d in days] == ["Пн", "Вт", "Ср", "Чт", "Пт"]


def test_start_weekday_wraps() -> None:
    """Тест: старт з неділі переходить на понеділок."""
    days = plan(SHIFTS, 2, start_weekday=6)
    assert days == [Day(1, "Нд", "A"), Day(2, "Пн", "B")]


def test_invalid_weekday() -> None:
    """Тест: некоректний день тижня - ValueError."""
    with pytest.raises(ValueError):
        weekdays_from(7)


def test_zero_days_and_empty_shifts() -> None:
    """Тест: n=0 і порожнє джерело змін."""
    assert plan(SHIFTS, 0) == []
    assert plan([], 5) == []


def test_schedule_is_infinite_but_lazy() -> None:
    """Тест: нескінченний розклад можна безпечно обмежити islice."""
    far = list(islice(cyclic_schedule(SHIFTS), 1000, 1001))
    assert far == [Day(1001, "Нд", "B")]


def test_laziness_counts_pulled_items() -> None:
    """Тест: з джерела змін береться рівно стільки, скільки потрібно."""
    pulled: list[str] = []
```

test_schedule.py рядки 58-98

```python
    def source() -> Iterator[str]:
        """Джерело змін, що запам'ятовує кожне звернення."""
        for shift in ("A", "B", "C", "D", "E"):
            pulled.append(shift)
            yield shift

    schedule = cyclic_schedule(source())
    assert pulled == []
    assert [d.shift for d in islice(schedule, 3)] == ["A", "B", "C"]
    assert pulled == ["A", "B", "C"]


def test_duty_roster() -> None:
    """Тест: працівники і зміни чергуються незалежно."""
    roster = list(islice(duty_roster(["X", "Y"], SHIFTS), 4))
    assert [(d.worker, d.shift) for d in roster] == [
        ("X", "A"),
        ("Y", "B"),
        ("X", "C"),
        ("Y", "A"),
    ]


def test_days_until() -> None:
    """Тест: takewhile до першої потрібної зміни."""
    assert len(days_until(SHIFTS, "C")) == 2
    assert days_until(SHIFTS, "A") == []
    assert len(days_until(SHIFTS, "Z", limit=10)) == 10


def test_weeks_chunked() -> None:
    """Тест: групування по 7 днів."""
    two = weeks(SHIFTS, 2)
    assert len(two) == 2 and all(len(w) == 7 for w in two)
    assert two[1][0].number == 8


def test_cumulative_hours() -> None:
    """Тест: накопичені години через accumulate."""
    hours = {"A": 8.0, "B": 10.0}
    assert cumulative_hours(SHIFTS, hours, 4) == [8.0, 18.0, 18.0, 26.0]
```

Файл tests/test_sliding_window.py

test_sliding_window.py рядки 1-57

```python
"""Тести sliding_window, moving_average та chunked."""

from collections.abc import Iterator
from itertools import count, starmap

import pytest

from streams import chunked, moving_average, sliding_window, take


def test_windows_basic() -> None:
    """Тест: базові вікна."""
    assert list(sliding_window([1, 2, 3, 4], 3)) == [(1, 2, 3), (2, 3, 4)]


def test_window_sums() -> None:
    """Тест: суми по вікнах через starmap."""
    sums = list(starmap(lambda a, b: a + b, sliding_window(range(5), 2)))
    assert sums == [1, 3, 5, 7]


def test_window_larger_than_source() -> None:
    """Тест: k більше за довжину - жодного вікна."""
    assert list(sliding_window([1, 2], 3)) == []
    assert list(sliding_window([], 1)) == []


def test_invalid_window() -> None:
    """Тест: некоректний розмір вікна."""
    with pytest.raises(ValueError):
        sliding_window([1, 2], 0)
    with pytest.raises(ValueError):
        chunked([1, 2], -1)


def test_moving_average() -> None:
    """Тест: рухоме середнє на нескінченному потоці."""
    assert take(3, moving_average(count(1), 2)) == [1.5, 2.5, 3.5]


def test_chunked() -> None:
    """Тест: пакети фіксованого розміру."""
    assert list(chunked(range(7), 3)) == [(0, 1, 2), (3, 4, 5), (6,)]


def test_window_laziness() -> None:
    """Тест: для одного вікна береться рівно k елементів."""
    pulled = []

    def numbers() -> Iterator[int]:
        """Нескінченне джерело, що рахує звернення."""
        for n in count():
            pulled.append(n)
            yield n

    assert take(1, sliding_window(numbers(), 4)) == [(0, 1, 2, 3)]
    assert pulled == [0, 1, 2, 3]
```

Файл tests/test_take_drop.py

test_take_drop.py рядки 1-36

```python
"""Тести take/drop."""

from itertools import count, islice

import pytest

from streams import after_non_positive, below, drop, take


def test_take_drop_on_count() -> None:
    """Тест: take просуває спільний ітератор."""
    xs = count(1)
    assert take(5, xs) == [1, 2, 3, 4, 5]
    assert take(3, xs) == [6, 7, 8]


def test_drop_then_take() -> None:
    """Тест: drop пропускає перші n."""
    assert list(islice(drop(10, count(0)), 3)) == [10, 11, 12]


def test_edge_cases() -> None:
    """Тест: n=0, порожнє джерело, від'ємні n."""
    assert take(0, count()) == []
    assert take(5, []) == []
    assert list(drop(3, [1, 2])) == []
    with pytest.raises(ValueError):
        take(-1, [1])
    with pytest.raises(ValueError):
        drop(-1, [1])


def test_takewhile_dropwhile() -> None:
    """Тест: умовні аналоги take/drop."""
    assert list(below(5, count(1))) == [1, 2, 3, 4]
    assert list(after_non_positive([-1, 0, 2, -3, 4])) == [2, -3, 4]
```

Файл tests/test_fib_stream.py

test_fib_stream.py рядки 1-15

```python
"""Тести потоку Фібоначчі."""

from itertools import islice

from streams import drop, fib_stream, take


def test_first_10() -> None:
    """Тест: перші 10 значень."""
    assert list(islice(fib_stream(), 10)) == [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]


def test_drop_then_take_fib() -> None:
    """Тест: пропустити 5, взяти 8."""
    assert take(8, drop(5, fib_stream())) == [5, 8, 13, 21, 34, 55, 89, 144]
```

Файл tests/test_accumulate.py

test_accumulate.py рядки 1-20

```python
"""Тести accumulate."""

import operator
from itertools import accumulate

from streams import prefix_products, prefix_sums


def test_accumulate_sum_prod() -> None:
    """Тест: префіксні суми і добутки."""
    assert list(accumulate([1, 2, 3, 4])) == [1, 3, 6, 10]
    assert list(accumulate([1, 2, 3, 4], operator.mul)) == [1, 2, 6, 24]
    assert list(prefix_sums([1, 2, 3, 4])) == [1, 3, 6, 10]
    assert list(prefix_products([1, 2, 3, 4])) == [1, 2, 6, 24]


def test_accumulate_max_and_empty() -> None:
    """Тест: префіксний максимум і порожній вхід."""
    assert list(accumulate([3, 1, 4, 1, 5], max)) == [3, 3, 4, 4, 5]
    assert list(prefix_sums([])) == []
```

Запуск програми

```
Windows PowerShell

PS D:\IPZ2kurs\func\lab1\Lab_05> py main.py
План на 10 днів (старт у середу):
  день  1 (Ср): Ранок
  день  2 (Чт): День
  день  3 (Пт): Ніч
  день  4 (Сб): Вихідний
  день  5 (Нд): Ранок
  день  6 (Пн): День
  день  7 (Вт): Ніч
  день  8 (Ср): Вихідний
  день  9 (Чт): Ранок
  день 10 (Пт): День

Графік чергувань на 6 днів:
  день 1: Іван - Ранок
  день 2: Олена - День
  день 3: Петро - Ніч
  день 4: Іван - Вихідний
  день 5: Олена - Ранок
  день 6: Петро - День

Днів до першої нічної зміни: 2
Накопичені години за 8 днів: [8.0, 16.0, 26.0, 26.0, 34.0, 42.0, 52.0, 52.0]
Перший тиждень: ['Ранок', 'День', 'Ніч', 'Вихідний', 'Ранок', 'День', 'Ніч']

Базові потоки:
  перші 10 Фібоначчі: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
  пропустити 5, взяти 8: [5, 8, 13, 21, 34, 55, 89, 144]
  вікна по 3: [(1, 2, 3), (2, 3, 4), (3, 4, 5)]
  рухоме середнє k=2: [1.5, 2.5, 3.5, 4.5]
```

Рис. 1. Вивід програми (python main.py)

Виконання тестів

```
Windows PowerShell

PS D:\IPZ2kurs\func\lab1\Lab_05> py -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.13.9, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\LENOVO\AppData\Local\Programs\Python\Python313\python.exe
rootdir: D:\IPZ2kurs\func\lab1\Lab_05
configfile: pyproject.toml
testpaths: tests
plugins: platformdirs-4.12.4
collecting ... collected 25 items

tests/test_accumulate.py::test_accumulate_sum_prod PASSED                [  4%]
tests/test_accumulate.py::test_accumulate_max_and_empty PASSED           [  8%]
tests/test_fib_stream.py::test_first_10 PASSED                           [ 12%]
tests/test_fib_stream.py::test_drop_then_take_fib PASSED                 [ 16%]
tests/test_schedule.py::test_plan_numbering_and_cycle PASSED             [ 20%]
tests/test_schedule.py::test_start_weekday_wraps PASSED                  [ 24%]
tests/test_schedule.py::test_invalid_weekday PASSED                      [ 28%]
tests/test_schedule.py::test_zero_days_and_empty_shifts PASSED           [ 32%]
tests/test_schedule.py::test_schedule_is_infinite_but_lazy PASSED        [ 36%]
tests/test_schedule.py::test_laziness_counts_pulled_items PASSED         [ 40%]
tests/test_schedule.py::test_duty_roster PASSED                          [ 44%]
tests/test_schedule.py::test_days_until PASSED                           [ 48%]
tests/test_schedule.py::test_weeks_chunked PASSED                        [ 52%]
tests/test_schedule.py::test_cumulative_hours PASSED                     [ 56%]
tests/test_sliding_window.py::test_windows_basic PASSED                  [ 60%]
tests/test_sliding_window.py::test_window_sums PASSED                    [ 64%]
tests/test_sliding_window.py::test_window_larger_than_source PASSED      [ 68%]
tests/test_sliding_window.py::test_invalid_window PASSED                 [ 72%]
tests/test_sliding_window.py::test_moving_average PASSED                 [ 76%]
tests/test_sliding_window.py::test_chunked PASSED                        [ 80%]
tests/test_sliding_window.py::test_window_laziness PASSED                [ 84%]
tests/test_take_drop.py::test_take_drop_on_count PASSED                  [ 88%]
tests/test_take_drop.py::test_drop_then_take PASSED                      [ 92%]
tests/test_take_drop.py::test_edge_cases PASSED                          [ 96%]
tests/test_take_drop.py::test_takewhile_dropwhile PASSED                 [100%]

============================== 25 passed in 0.08s ==============================
```

Рис. 2. Результат запуску pytest

Перевірка типів і стилю

```
Windows PowerShell

PS D:\IPZ2kurs\func\lab1\Lab_05> py -m mypy .
Success: no issues found in 8 source files
PS D:\IPZ2kurs\func\lab1\Lab_05> py -m ruff check .
All checks passed!
PS D:\IPZ2kurs\func\lab1\Lab_05> py -m black --check .
All done!
8 files would be left unchanged.
```

Рис. 3. mypy, ruff та black без зауважень

Висновок:

Під час виконання роботи побудовано лінивий нескінченний циклічний розклад на основі cycle, count, zip та islice, а також набір базових лінивих утиліт. Значення обчислюються тільки тоді, коли їх запитують, тому з нескінченними потоками можна працювати безпечно, обмежуючи їх take, islice або takewhile. Лінивість підтверджено тестами з лічильником звернень до джерела.